

ISIS-1611: Inteligencia Artificial

Notas del Curso Semestre 2026-19

Universidad de los Andes

Basado en: *Artificial Intelligence: A Modern Approach*

Russell, Norvig & Ming-Wei Chang (4^a ed.)

Carla González

8 de julio de 2026

Índice general

I	Búsqueda	5
1.	Semana 1: Introducción y Búsqueda Desinformada	6
1.1.	Contexto e Historia de la IA	6
1.1.1.	Impacto de la IA en el mundo real	6
1.1.2.	¿Qué es la Inteligencia Artificial?	6
1.1.3.	Historia de la IA	6
1.2.	Agentes Inteligentes	6
1.2.1.	¿Qué es un Agente?	6
1.2.2.	Racionalidad	7
1.2.3.	Descripción PEAS	7
1.2.4.	Naturaleza de los Ambientes	7
1.2.5.	Tipos de Agentes	8
1.3.	Búsqueda Desinformada I: Formulación del Problema	8
1.3.1.	Formulación de un Problema de Búsqueda	8
1.3.2.	Nodos vs. Estados	9
1.3.3.	Medidas de Desempeño de Algoritmos	9
1.4.	Búsqueda Desinformada II: Algoritmos	9
1.4.1.	Breadth-First Search (BFS)	9
1.4.2.	Depth-First Search (DFS)	9
1.4.3.	Depth-Limited Search y Iterative Deepening (IDDFS)	10
1.4.4.	Uniform-Cost Search (UCS)	10
1.4.5.	Comparativa Búsqueda Desinformada	10
1.5.	Búsqueda Informada: Heurísticas	10
1.5.1.	Función Heurística	10
1.5.2.	Greedy Best-First Search	10
1.5.3.	A*	11
2.	Semana 2: Búsqueda Informada y Ambientes Complejos	12
2.1.	Búsqueda Informada Avanzada	12
2.1.1.	Búsqueda con Memoria Acotada	12
2.1.2.	Heurísticas: Dominancia y Creación	13
2.2.	Búsqueda en Ambientes Complejos	13
2.2.1.	Búsqueda Local	13
2.2.2.	Hill Climbing (Ascenso de Colina)	13
2.2.3.	Simulated Annealing	14
2.2.4.	Algoritmos Genéticos	14
2.3.	Ejercicios de Búsqueda	14
2.3.1.	Guía de Ejercicios — Resumen de Bloques	14

II	Razonamiento bajo Incertidumbre	16
3.	Semana 3: Búsqueda Adversaria y Razonamiento bajo Incertidumbre	17
3.1.	Búsqueda Adversaria y Juegos I	17
3.1.1.	Juegos como Problemas de Búsqueda	17
3.1.2.	Minimax	17
3.2.	Búsqueda Adversaria y Juegos II	18
3.2.1.	Poda Alfa-Beta (α - β)	18
3.2.2.	Juegos con Incertidumbre: Expectiminimax	18
3.2.3.	Funciones de Evaluación	19
3.2.4.	Monte Carlo Tree Search (MCTS)	19
3.3.	Razonamiento bajo Incertidumbre I	19
3.3.1.	Probabilidad como Base del Razonamiento	19
3.3.2.	Regla de Bayes	19
3.3.3.	Distribuciones de Probabilidad	20
3.4.	Razonamiento bajo Incertidumbre II	20
3.4.1.	Inferencia por Enumeración	20
3.4.2.	Variables y Modelos	20
4.	Semana 4: Redes Bayesianas, MDPs y CSP	22
4.1.	Redes Bayesianas	22
4.1.1.	Semántica	22
4.1.2.	Inferencia Exacta	22
4.1.3.	Inferencia Aproximada	22
4.2.	Procesos de Decisión de Markov (MDPs)	23
4.2.1.	Propiedad de Markov	23
4.2.2.	Política y Utilidad	24
4.2.3.	Ecuación de Bellman	24
4.2.4.	Algoritmos de Solución	24
4.3.	Ejercicios: Razonamiento bajo Incertidumbre	24
4.4.	Problemas de Satisfacción de Restricciones (CSP)	25
4.4.1.	Tipos de Restricciones	25
4.4.2.	Búsqueda con Retroceso (Backtracking)	25
5.	Semana 5: CSP Avanzado y Agentes Lógicos	27
5.1.	Problemas de Satisfacción de restricciones	27
5.1.1.	Diagrama de Arbol	27
5.1.2.	k-consistencia	28
5.1.3.	Forward Checking	28
5.1.4.	Estructura del grafo de restricciones	28
5.1.5.	Búsqueda local para CSP: Min-conflicts	29
5.2.	Ejercicio: Coloreado del mapa de Australia	29
5.3.	Agentes Lógicos: Introducción	30
5.3.1.	Agentes basados en conocimiento	30
5.3.2.	Lógica proposicional	31
5.3.3.	Formas normales: CNF y DNF	31
5.3.4.	El mundo del Wumpus	31

III	Lógica y Representación del Conocimiento	33
6.	Semana 6: Lógica y Representación del Conocimiento	34
6.1.	Agentes Lógicos II: Inferencia Proposicional	34
6.1.1.	Reglas de Inferencia	34
6.1.2.	Resolución	34
6.1.3.	DPLL y WalkSAT	34
6.1.4.	Ejemplo: Refutación por Resolución	34
6.2.	Lógica de Primer Orden (LPO)	35
6.2.1.	Sintaxis	35
6.2.2.	Semántica	35
6.2.3.	Cuantificadores	36
6.2.4.	Ejemplo: Modelo en LPO	36
6.3.	LPO e Inferencia I	36
6.3.1.	Bases de Conocimiento en LPO	36
6.3.2.	Unificación	36
6.3.3.	Inferencia con Cuantificadores	37
6.3.4.	Forma Normal de Clausura (CNF en LPO)	37
6.3.5.	Ejemplo: Unificación	37
6.4.	Inferencia en LPO II	37
6.4.1.	Encadenamiento hacia adelante (Forward Chaining)	37
6.4.2.	Encadenamiento hacia atrás (Backward Chaining)	38
6.4.3.	Resolución en LPO	38
6.4.4.	Ejemplo: El caso de Nono (¿Es West un criminal?)	38
IV	Planeación	40
7.	Semana 7: Inferencia en LPO y Planeación	41
7.1.	Inferencia en LPO: Cierre	41
7.2.	Planeación: Definición y Algoritmos Generales	41
7.2.1.	¿Qué es la Planeación?	41
7.2.2.	Representación PDDL	41
7.2.3.	Algoritmos de Búsqueda para Planeación	42
7.3.	Planeación: Heurísticas y Planeación Jerárquica	42
7.3.1.	Heurísticas para Planeación	42
7.3.2.	Planeación Jerárquica (HTN)	42
7.4.	Planeación en Ambientes No Deterministas	42
7.4.1.	Ambientes No Deterministas y Condicionales	42
7.4.2.	Planeación Parcialmente Observable	42
7.4.3.	Planeación Continua y de Ejecución	43
V	IA Moderna, Aplicaciones y Ética	44
8.	Semana 8: Aplicaciones, Ética y Repaso Final	45
8.1.	Aplicaciones en IA	45
8.1.1.	Machine Learning	45
8.1.2.	Deep Learning	45
8.1.3.	Áreas de Aplicación	45
8.2.	Ética en IA y Aplicaciones Futuras	46
8.2.1.	Riesgos y Desafíos Éticos	46

8.2.2.	Marcos Regulatorios	46
8.2.3.	Niveles de Uso de IA Generativa (Uniandes)	46
8.2.4.	Futuro de la IA	46
8.3.	Repaso para Examen Final	46
8.3.1.	Resumen de Temas	46

Parte I

Búsqueda

Capítulo 1

Semana 1: Introducción y Búsqueda Desinformada

1.1. Contexto e Historia de la IA

1.1.1. Impacto de la IA en el mundo real

La IA impacta múltiples sectores: economía, política, ley y derecho, trabajo, ciencia, salud y educación.

1.1.2. ¿Qué es la Inteligencia Artificial?

Existen cuatro enfoques para definir la IA:

Pensamiento humano Modelos cognitivos	Pensamiento racional Leyes del pensamiento (lógica)
Actuar humanamente Test de Turing	Actuar racionalmente Agentes racionales

Nota 1.1.1. La IA moderna se enfoca en **agentes racionales**: entidades que perciben su entorno y actúan para maximizar su medida de desempeño esperada.

1.1.3. Historia de la IA

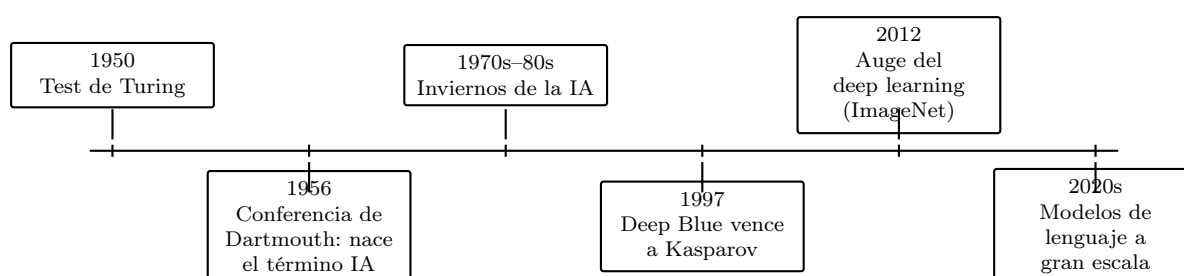


Figura 1.1: Línea de tiempo con algunos hitos de la historia de la IA.

1.2. Agentes Inteligentes

1.2.1. ¿Qué es un Agente?

Definición 1.2.1 (Agente). Un agente es cualquier entidad que **percibe** su ambiente mediante **sensores** y **actúa** sobre él mediante **actuadores**.

La función del agente mapea percepciones a acciones: $f : P^* \rightarrow A$

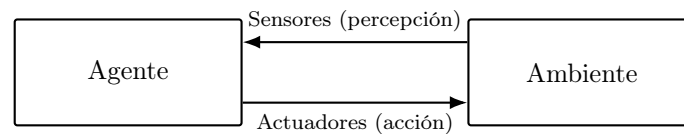


Figura 1.2: Ciclo agente-ambiente: el agente percibe el ambiente a través de sensores y actúa sobre él mediante actuadores, en un ciclo continuo.

Ejemplo 1.2.1 (PEAS de un taxi autónomo). La descripción PEAS de un taxi autónomo es:

- **Performance:** seguridad, tiempo de viaje, cumplir normas de tránsito, comodidad del pasajero.
- **Environment:** calles, otros vehículos, peatones, señales de tránsito, clima.
- **Actuators:** dirección, acelerador, freno, señales, bocina.
- **Sensors:** cámaras, GPS, velocímetro, sensores de proximidad (LIDAR/radar).

1.2.2. Racionalidad

Un agente racional selecciona la acción que maximiza su **medida de desempeño** esperada, dado su conocimiento actual.

Depende de:

1. La medida de desempeño
2. El conocimiento previo del entorno
3. Las acciones disponibles
4. La secuencia de percepciones hasta el momento

1.2.3. Descripción PEAS

Para caracterizar el ambiente de una tarea se usa PEAS:

- **Performance** (Medida de desempeño)
- **Environment** (Entorno)
- **Actuators** (Actuadores)
- **Sensors** (Sensores)

1.2.4. Naturaleza de los Ambientes

Dimensión	Valores posibles
Observabilidad	Completamente observable / Parcialmente observable
Agentes	Un solo agente / Multiagente
Determinismo	Determinista / Estocástico
Episodicidad	Episódico / Secuencial
Dinamismo	Estático / Dinámico
Tiempo	Discreto / Continuo
Conocimiento	Conocido / Desconocido

1.2.5. Tipos de Agentes

1. **Agente de reflejo simple:** actúa según la percepción actual (reglas condición-acción).
2. **Agente de reflejo basado en modelos:** mantiene un estado interno del mundo.
3. **Agente basado en objetivos:** considera metas para elegir acciones.
4. **Agente basado en utilidad:** maximiza una función de utilidad.
5. **Agente que aprende:** mejora su desempeño con la experiencia.

1.3. Búsqueda Desinformada I: Formulación del Problema

1.3.1. Formulación de un Problema de Búsqueda

Definición 1.3.1 (Problema de búsqueda). Un problema de búsqueda se define con cinco componentes:

1. **Estado inicial:** s_0
2. **Modelo de transición:** $\text{RESULT}(s, a) \rightarrow s'$
3. **Función de costo de acción:** $c(s, a, s')$
4. **Test de meta:** $\text{IS-GOAL}(s) \rightarrow \{T, F\}$
5. **Espacio de estados:** grafo de todos los estados alcanzables

La **solución** es una secuencia de acciones que lleva del estado inicial a un estado meta. Una **solución óptima** minimiza el costo total del camino.

Ejemplo 1.3.1 (Mapa de Rumania). Un viajero está en la ciudad de Arad y debe llegar a Bucarest.

- **Estado inicial:** *Arad*.
- **Modelo de transición:** $\text{RESULT}(\text{Arad}, \text{ir a Zerind}) = \text{Zerind}$, y así para cada carretera.
- **Costo de acción:** la distancia en kilómetros de cada carretera.
- **Test de meta:** estado = *Bucarest*.
- **Espacio de estados:** el grafo de ciudades rumanas conectadas por carreteras.

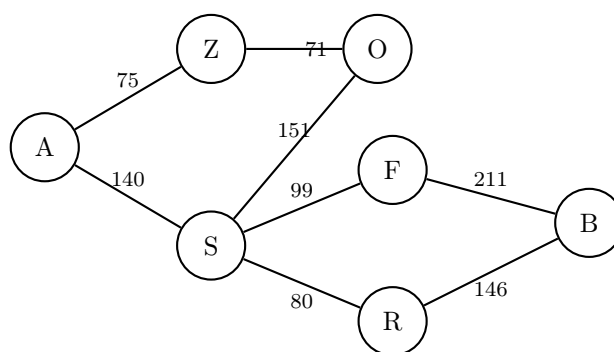


Figura 1.3: Fragmento del espacio de estados del mapa de Rumania: nodos = ciudades, aristas = carreteras con su costo (distancia en km). Este grafo es la representación del *problema*; el árbol de búsqueda se construye explorándolo a partir de *Arad*.

1.3.2. Nodos vs. Estados

Un **nodo** en el árbol de búsqueda contiene:

- El estado que representa
- El nodo padre
- La acción que generó el nodo
- El costo del camino $g(n)$
- La profundidad

1.3.3. Medidas de Desempeño de Algoritmos

Criterio	Descripción
Completitud	¿Encuentra solución si existe?
Optimalidad	¿Encuentra la solución de menor costo?
Complejidad en tiempo	Nodos generados/expandidos
Complejidad en espacio	Nodos en memoria simultáneamente

Los parámetros usuales son b (factor de ramificación), d (profundidad de la solución más superficial) y m (profundidad máxima del árbol).

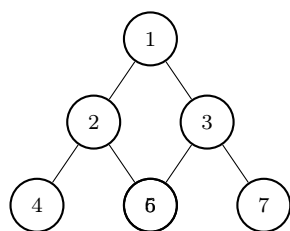
1.4. Búsqueda Desinformada II: Algoritmos

1.4.1. Breadth-First Search (BFS)

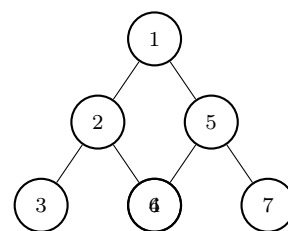
- Expande el nodo más **superficial** primero (FIFO).
- **Completo**: sí (si b finito).
- **Óptimo**: sí, si todos los costos de paso son iguales.
- Tiempo: $\mathcal{O}(b^d)$ Espacio: $\mathcal{O}(b^d)$

1.4.2. Depth-First Search (DFS)

- Expande el nodo más **profundo** primero (LIFO).
- **Completo**: no en árbol infinito; sí en grafo finito.
- **Óptimo**: no.
- Tiempo: $\mathcal{O}(b^m)$ Espacio: $\mathcal{O}(bm)$



BFS: orden de expansión 1,2,3,4,5,6,7 (nivel por nivel).



DFS: orden de expansión 1,2,3,4,5,6,7 (baja hasta el fondo antes de retroceder).

Figura 1.4: Mismo árbol de búsqueda, dos órdenes de expansión distintos según la estructura de datos de la frontera (cola FIFO en BFS, pila LIFO en DFS).

1.4.3. Depth-Limited Search y Iterative Deepening (IDDFS)

IDDFS ejecuta DFS con límite de profundidad creciente $l = 0, 1, 2, \dots$

- Combina las ventajas de BFS (completo, óptimo si costos iguales) con las de DFS (espacio lineal).
- Tiempo: $\mathcal{O}(b^d)$ Espacio: $\mathcal{O}(bd)$

1.4.4. Uniform-Cost Search (UCS)

- Expande el nodo con menor **costo de camino** $g(n)$ (cola de prioridad).
- Equivale a BFS cuando todos los costos son iguales.
- **Completo**: sí (si costos $> \epsilon > 0$).
- **Óptimo**: sí.
- Tiempo y espacio: $\mathcal{O}(b^{1+\lceil C^*/\epsilon \rceil})$, donde C^* es el costo óptimo.

Ejemplo 1.4.1 (UCS sobre un grafo con costos). Usando el grafo de Rumania de la clase anterior, para ir de *Arad* a *Bucarest*, UCS explora en este orden (por costo acumulado $g(n)$ creciente): *Arad*(0), *Zerind*(75), *Sibiu*(140), *Oradea*(146), *Rimnicu*(220), *Fagaras*(239), *Bucarest* vía *Rimnicu* $\rightarrow$ *Pitesti* $\rightarrow$ *Bucarest* con costo 418, que resulta más barato que ir vía *Fagaras* (costo 450), aunque *Fagaras* se alcance primero.

1.4.5. Comparativa Búsqueda Desinformada

Algoritmo	Completo	Óptimo	Tiempo	Espacio
BFS	Sí	Sí*	b^d	b^d
DFS	No	No	b^m	bm
IDDFS	Sí	Sí*	b^d	bd
UCS	Sí	Sí	$b^{C^*/\epsilon}$	$b^{C^*/\epsilon}$

*Óptimo si costos de paso son iguales

1.5. Búsqueda Informada: Heurísticas

1.5.1. Función Heurística

Definición 1.5.1 (Heurística). $h(n)$ es una función que estima el costo desde el nodo n hasta la meta más cercana. Debe ser **admisible**: nunca sobreestima el costo real.

Nota 1.5.1. Una heurística es **admisible** si $h(n) \leq h^*(n)$ para todo n , donde $h^*(n)$ es el costo real al llegar a la meta.

1.5.2. Greedy Best-First Search

Expande el nodo que parece más cercano a la meta según $h(n)$.

- Función de evaluación: $f(n) = h(n)$
- No es **óptimo** ni completo en general.
- Rápido en la práctica, puede quedar atrapado en mínimos locales.

1.5.3. A*

Combina el costo real del camino y la heurística:

$$f(n) = g(n) + h(n)$$

- $g(n)$: costo del camino desde inicio hasta n
- $h(n)$: estimación del costo desde n hasta la meta
- **Completo y óptimo** si h es admisible (búsqueda en árbol) o consistente (búsqueda en grafo).

Definición 1.5.2 (Consistencia / Monotonía). h es consistente si para todo nodo n y sucesor n' :

$$h(n) \leq c(n, a, n') + h(n')$$

La consistencia implica admisibilidad.

Ejemplo 1.5.1 (Heurísticas comunes en una cuadrícula). Para un agente que se mueve en una cuadrícula 2D desde (x_1, y_1) hasta (x_2, y_2) :

- **Distancia Manhattan:** $h(n) = |x_1 - x_2| + |y_1 - y_2|$. Admisible si el agente solo se mueve en horizontal/vertical (no sobreestima, porque es exactamente el número mínimo de pasos sin obstáculos).
- **Distancia euclidiana:** $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. Admisible si se permite movimiento diagonal, ya que nunca es mayor que el camino real.

Usar Manhattan cuando solo hay 4 movimientos posibles evita sobreestimar; usarla con movimiento diagonal permitiría violar la admisibilidad, porque el camino real podría ser más corto que la suma de catetos.

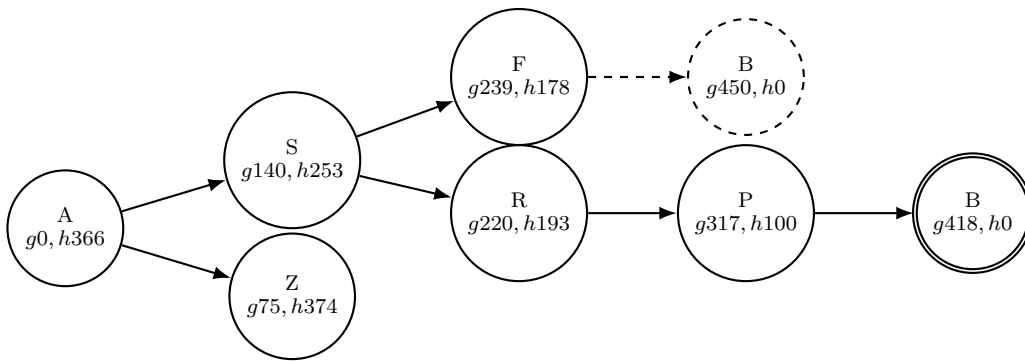


Figura 1.5: Trazo de A* sobre el mapa de Rumania (heurística: distancia en línea recta a Bucarest). A* siempre expande el nodo de menor $f(n) = g(n) + h(n)$ de la frontera. Aunque Fagaras (F) llega primero a Bucarest con $f = 450$ (nodo punteado: se genera pero **se descarta**), Rimnicu (R) tiene $f = 220 + 193 = 413$, menor que $f(F) = 239 + 178 = 417$, así que A* explora primero $R \rightarrow$ Pitesti (P) con $f = 317 + 100 = 417$. Al final, la meta alcanzada por Pitesti tiene $f = 418$, menor que los 450 de la rama de Fagaras, así que esa es la **solución óptima** (nodo de doble borde).

Capítulo 2

Semana 2: Búsqueda Informada y Ambientes Complejos

Nota 2.0.1 (Eventos esta semana). ■ Publicación **Taller 1**

- **Examen 1** (al final de la semana)

2.1. Búsqueda Informada Avanzada

2.1.1. Búsqueda con Memoria Acotada

Iterative Deepening A* (IDA*)

Versión de A* que usa IDDFS con el valor de corte $f(n) = g(n) + h(n)$.

- Espacio: $\mathcal{O}(bd)$
- Completo y óptimo (con h admisible)

Recursive Best-First Search (RBFS)

Mantiene el mejor valor alternativo f en cada nivel de recursión.

- Espacio: $\mathcal{O}(bd)$
- Puede reexpandir nodos (regenera sub-árboles olvidados)

SMA* (Simplified Memory-Bounded A*)

Usa toda la memoria disponible; descarta el nodo con mayor f cuando la memoria está llena.

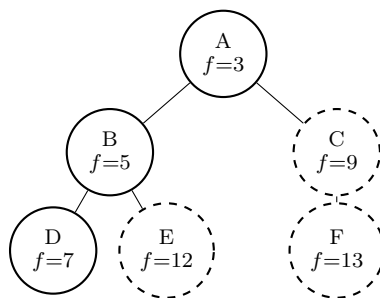


Figura 2.1: IDA* con umbral inicial $f \leq 7$ (el valor de h en la raíz): en la primera iteración se expanden A , B y D (todos con $f \leq 7$), y se podan C , E y F (nodos punteados, con $f > 7$). Si no se halla la meta, el umbral sube al menor f podado (aquí 9) y se repite la búsqueda desde cero.

2.1.2. Heurísticas: Dominancia y Creación

Definición 2.1.1 (Dominancia). h_2 domina a h_1 si $h_2(n) \geq h_1(n)$ para todo n y ambas son admisibles. Una heurística dominante expande menos nodos.

Formas de construir heurísticas admisibles:

1. **Relajación del problema:** eliminar restricciones del problema original.
2. **Bases de datos de patrones:** precomputar costos exactos de subproblemas.
3. **Máximo de heurísticas:** $h(n) = \max(h_1(n), h_2(n))$

Ejemplo 2.1.1 (8-puzzle: dos heurísticas admisibles). Sea n un estado del 8-puzzle donde las fichas 1, 2 y 3 no están en su casilla objetivo.

- $h_1(n)$ = número de **fichas fuera de lugar**. Si 3 fichas están mal ubicadas, $h_1(n) = 3$.
- $h_2(n)$ = suma de **distancias Manhattan** de cada ficha a su casilla objetivo. Si esas 3 fichas están, respectivamente, a 1, 2 y 3 movimientos de su lugar, $h_2(n) = 1 + 2 + 3 = 6$.

Ambas son admisibles (relajaciones del problema: h_1 ignora la restricción de que una ficha se mueva solo a una casilla adyacente vacía; h_2 ignora que solo puede moverse una ficha a la vez). Como $h_2(n) \geq h_1(n)$ siempre, h_2 **domina** a h_1 y expande menos nodos en A*/IDA*.

2.2. Búsqueda en Ambientes Complejos

2.2.1. Búsqueda Local

A diferencia de la búsqueda sistemática, la búsqueda local opera sobre un **estado actual** sin mantener el camino recorrido. Útil cuando el objetivo es el estado mismo (no el camino).

2.2.2. Hill Climbing (Ascenso de Colina)

Hill climbing mantiene un único estado actual y se mueve al vecino con mejor valor, deteniéndose cuando ningún vecino mejora (un óptimo local respecto al movimiento permitido).

Problemas:

- Máximos locales (no globales)
- Mesetas (*plateaus*) y crestas

Variantes: Hill climbing estocástico, reinicio aleatorio, *simulated annealing*.

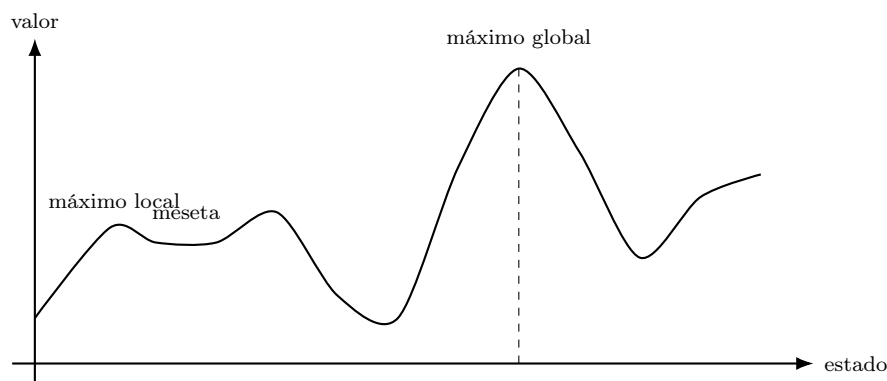


Figura 2.2: Paisaje de valores de estados: hill climbing puede quedar atrapado en el máximo local de la izquierda o en la meseta, sin llegar nunca al máximo global de la derecha, porque solo acepta movimientos que mejoran el valor actual.

2.2.3. Simulated Annealing

Permite movimientos malos con probabilidad $e^{\Delta E/T}$ donde T decrece con el tiempo. Garantiza convergencia al óptimo global si T baja suficientemente despacio.

Ejemplo 2.2.1 (Aceptar un movimiento malo). Si un movimiento empeora el valor en $\Delta E = -2$ y la temperatura actual es $T = 1$, la probabilidad de aceptarlo de todos modos es $e^{\Delta E/T} = e^{-2/1} \approx 0,135$ (13.5 %). Con $T = 4$ (temperatura más alta, típico al inicio), esa misma probabilidad sube a $e^{-2/4} \approx 0,607$ (60.7 %): al principio el algoritmo explora más libremente escapando de máximos locales; al final ($T \rightarrow 0$) se comporta como hill climbing puro.

2.2.4. Algoritmos Genéticos

Inspirados en la evolución biológica. Operan sobre una **población** de individuos (estados codificados como cadenas).

Operadores:

1. **Selección**: individuos con mayor aptitud (*fitness*) tienen más probabilidad de reproducirse.
2. **Cruzamiento** (*crossover*): combina dos individuos para producir descendencia.
3. **Mutación**: cambia aleatoriamente genes con baja probabilidad.

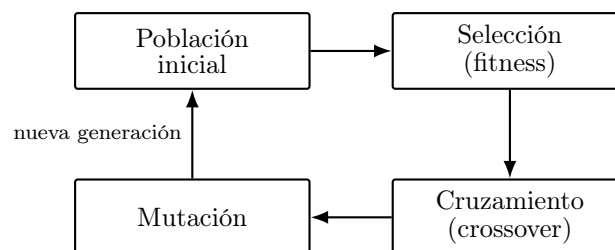


Figura 2.3: Ciclo de un algoritmo genético: de la población actual se seleccionan padres según su aptitud, se cruzan para producir hijos, se mutan con baja probabilidad, y la nueva generación reemplaza a la anterior.

Ejemplo 2.2.2 (Cruzamiento de un 8-reinas codificado). Si dos individuos del problema de las 8 reinas se codifican como cadenas de 8 dígitos (la fila de la reina en cada columna), por ejemplo 32752411 y 24748552, un cruzamiento de un punto tras la posición 3 produce el hijo 327 (del primero) + 48552 (del segundo) = 32748552, combinando la parte inicial de un padre con la parte final del otro.

2.3. Ejercicios de Búsqueda

Nota 2.3.1. Esta clase es de ejercicios y repaso para el **Examen 1**. Ver la *Guía de Ejercicios: Búsqueda Desinformada e Informada*.

2.3.1. Guía de Ejercicios — Resumen de Bloques

Bloque 1: Agentes Inteligentes

Caracterizar escenarios con PEAS, identificar tipo de agente y entorno.

Bloque 2: Formulación de Problemas

Definir estado inicial, modelo de transición, costo y test de meta para problemas concretos.

Bloque 3: Búsqueda No Informada

Aplicar BFS, DFS y UCS sobre grafos; analizar completitud y optimalidad.

Bloque 4: Búsqueda Informada y Heurísticas

Aplicar Greedy, A*, RBFS, SMA*; verificar admisibilidad y consistencia de heurísticas.

Ejercicio 2.3.1. Dado el grafo del Bloque 3 (Guía), aplica BFS, DFS y UCS. Indica el orden de expansión y el camino solución encontrado.

Solución:

Parte II

Razonamiento bajo Incertidumbre

Capítulo 3

Semana 3: Búsqueda Adversaria y Razonamiento bajo Incertidumbre

Nota 3.0.1 (Eventos esta semana). ■ **Entrega Taller 1**

3.1. Búsqueda Adversaria y Juegos I

3.1.1. Juegos como Problemas de Búsqueda

Un **juego** de dos jugadores, suma cero, de información perfecta se define como:

- S_0 : estado inicial
- $\text{PLAYER}(s)$: qué jugador mueve en estado s
- $\text{ACTIONS}(s)$: movimientos legales
- $\text{RESULT}(s, a)$: estado resultante
- $\text{IS-TERMINAL}(s)$: test de fin de juego
- $\text{UTILITY}(s, p)$: valor numérico del estado terminal para el jugador p

3.1.2. Minimax

MAX maximiza, MIN minimiza la utilidad.

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s, \text{MAX}) & \text{si IS-TERMINAL}(s) \\ \max_a \text{MINIMAX}(\text{RESULT}(s, a)) & \text{si PLAYER}(s) = \text{MAX} \\ \min_a \text{MINIMAX}(\text{RESULT}(s, a)) & \text{si PLAYER}(s) = \text{MIN} \end{cases}$$

- Completo (en árbol finito) y óptimo contra un oponente óptimo.
- Tiempo: $\mathcal{O}(b^m)$ Espacio: $\mathcal{O}(bm)$

Ejemplo 3.1.1 (Minimax sobre un árbol de profundidad 2). En el árbol de la Figura siguiente, cada hoja es un estado terminal con su utilidad para MAX. Los valores se calculan de abajo hacia arriba: cada nodo MIN toma el mínimo de sus hijos, y la raíz (MAX) toma el máximo de los valores que le devuelven los nodos MIN.

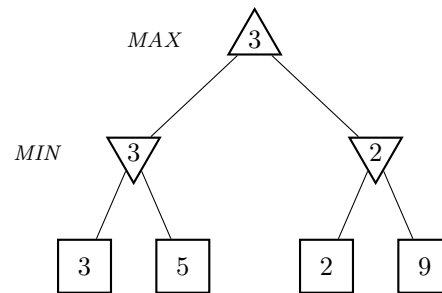


Figura 3.1: Árbol minimax: los nodos MIN eligen el menor de sus hijos ($\min(3, 5) = 3$ y $\min(2, 9) = 2$); la raíz MAX elige el mayor de esos valores ($\max(3, 2) = 3$), así que la rama izquierda es la jugada óptima para MAX.

3.2. Búsqueda Adversaria y Juegos II

3.2.1. Poda Alfa-Beta (α - β)

Elimina ramas del árbol que no pueden afectar la decisión final.

- α : mejor valor que MAX puede garantizar (lo que MAX ya tiene)
- β : mejor valor que MIN puede garantizar (lo que MIN ya tiene)
- Poda: si $v \geq \beta$ en nodo MAX $\Rightarrow$ cortar; si $v \leq \alpha$ en nodo MIN $\Rightarrow$ cortar.

Con orden perfecto de movimientos: $\mathcal{O}(b^{m/2})$ (duplica la profundidad manejable).

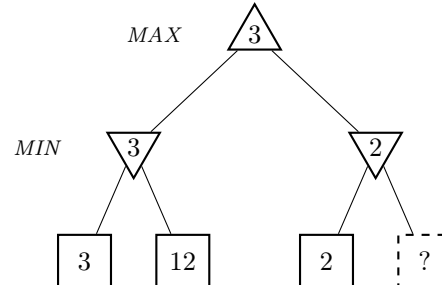


Figura 3.2: Poda alfa-beta: tras explorar la rama izquierda, MAX ya garantiza al menos $\alpha = 3$. Al explorar la rama derecha, MIN encuentra un hijo con valor 2, así que ya sabe que esa rama vale $\leq 2 < \alpha$: como MAX nunca elegiría un valor menor que 3 pudiendo garantizarlo, el segundo hijo de B (nodo punteado) se **poda** sin evaluarlo, sin cambiar la decisión final.

3.2.2. Juegos con Incertidumbre: Expectiminimax

Para juegos con elementos aleatorios (dados) se agregan **nodos azar**:

$$\text{EXPECTIMINIMAX}(s) = \sum_r P(r) \cdot \text{EXPECTIMINIMAX}(\text{RESULT}(s, r))$$

Ejemplo 3.2.1 (Nodo de azar con una moneda). Si en un nodo de azar una moneda justa decide entre dos resultados con utilidades 8 y 2, el valor del nodo de azar es $0,5 \cdot 8 + 0,5 \cdot 2 = 5$, sin importar si el nodo padre es MAX o MIN: el valor esperado se calcula *antes* de que el jugador siguiente decida.

3.2.3. Funciones de Evaluación

En juegos reales no se puede explorar hasta el final. Se usa una **función de evaluación heurística** $EVAL(s)$ que estima la utilidad desde un estado no terminal.

Características de una buena función de evaluación:

- Debe concordar con la utilidad real en estados terminales.
- Rápida de calcular.
- Debe reflejar las probabilidades reales de ganar.

3.2.4. Monte Carlo Tree Search (MCTS)

Estrategia basada en simulaciones aleatorias (*rollouts*). Cuatro fases: selección, expansión, simulación, retropropagación.

3.3. Razonamiento bajo Incertidumbre I

3.3.1. Probabilidad como Base del Razonamiento

Cuando el entorno es **parcialmente observable** o **estocástico**, se usa probabilidad para representar la incertidumbre.

Definición 3.3.1 (Probabilidad a priori y condicional). La **probabilidad a priori** (o incondicional) $P(A)$ representa el grado de creencia sin ninguna evidencia adicional.

La **probabilidad condicional** $P(A | B) = \frac{P(A \wedge B)}{P(B)}$ representa el grado de creencia dado que B es verdadero.

3.3.2. Regla de Bayes

$$P(A | B) = \frac{P(B | A) \cdot P(A)}{P(B)}$$

- $P(A)$: probabilidad a priori de A (hipótesis)
- $P(B | A)$: verosimilitud de la evidencia B dado A
- $P(B)$: probabilidad marginal de B (normalizador)
- $P(A | B)$: probabilidad a posteriori

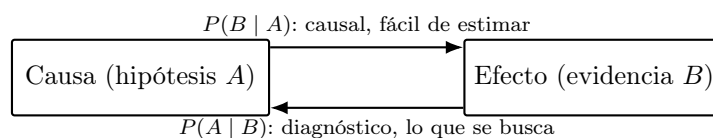


Figura 3.3: La regla de Bayes permite invertir el sentido del razonamiento: de un modelo *causal* $P(\text{efecto} | \text{causa})$, que suele conocerse de antemano (estadísticas, estudios clínicos), se obtiene la probabilidad *diagnóstica* $P(\text{causa} | \text{efecto})$, que es la que realmente interesa inferir.

Ejemplo 3.3.1 (Diagnóstico médico). Sea $M = \text{tiene meningitis}$ y $R = \text{tiene rigidez de cuello}$. Se sabe que $P(R | M) = 0,7$ (la meningitis causa rigidez de cuello el 70 % de las veces), $P(M) = 1/50000$ (prevalencia a priori) y $P(R) = 0,01$ (rigidez de cuello en la población general). Por la regla de Bayes:

$$P(M | R) = \frac{P(R | M) \cdot P(M)}{P(R)} = \frac{0,7 \times \frac{1}{50000}}{0,01} = 0,0014$$

Aunque la rigidez de cuello es un síntoma fuerte de meningitis (70 %), la probabilidad posterior sigue siendo baja porque la meningitis es muy rara a priori ($P(M)$ pequeño) y la rigidez de cuello es relativamente común por otras causas ($P(R)$ no tan pequeño).

3.3.3. Distribuciones de Probabilidad

Para variables aleatorias discretas, la **distribución conjunta completa** especifica $P(X_1, X_2, \dots, X_n)$ para todos los valores posibles. De ella se puede derivar cualquier probabilidad por marginalización y condicionamiento.

Marginalización: $P(X) = \sum_y P(X, Y = y)$

Independencia: $X \perp Y \Leftrightarrow P(X, Y) = P(X) \cdot P(Y)$

Independencia condicional: $X \perp Y | Z \Leftrightarrow P(X | Y, Z) = P(X | Z)$

3.4. Razonamiento bajo Incertidumbre II

3.4.1. Inferencia por Enumeración

Para cualquier consulta $P(X | \mathbf{e})$:

$$P(X | \mathbf{e}) = \alpha \sum_{\mathbf{y}} P(X, \mathbf{e}, \mathbf{y})$$

donde $\mathbf{y}$ son las variables ocultas y α es el normalizador.

3.4.2. Variables y Modelos

- **Variable aleatoria:** puede ser discreta o continua.
- **Distribución de probabilidad:** asigna probabilidades a todos los valores.
- **Tabla de probabilidad condicional (CPT):** especifica $P(X | \text{Parents}(X))$.

Ejemplo 3.4.1 (Enumeración sobre Dolor de muela, Caries y Detector). Sea la distribución conjunta completa $P(\text{Caries}, \text{Dolor}, \text{Detector})$:

	Dolor		¬Dolor	
	Detector	¬Detector	Detector	¬Detector
Caries	0.108	0.012	0.072	0.008
¬Caries	0.016	0.064	0.144	0.576

Cuadro 3.1: Distribución conjunta completa (suma de las 8 celdas = 1).

Para calcular $P(\text{Caries} | \text{Dolor})$ por enumeración, se **suma sobre la variable oculta** Detector, dejando fija la evidencia Dolor:

$$P(\text{Caries} | \text{Dolor}) = \alpha \sum_{\text{det}} P(\text{Caries}, \text{Dolor}, \text{det}) = \alpha (0,108 + 0,012) = \alpha (0,12)$$

De forma análoga, $P(\neg \text{Caries} \mid \text{Dolor}) = \alpha(0,016 + 0,064) = \alpha(0,08)$. Normalizando con $\alpha = 1/(0,12+0,08) = 5$: $P(\text{Caries} \mid \text{Dolor}) = 0,6$ y $P(\neg \text{Caries} \mid \text{Dolor}) = 0,4$. Este es el patrón general de inferencia por enumeración: fijar la evidencia, sumar sobre las variables ocultas, y normalizar al final.

Capítulo 4

Semana 4: Redes Bayesianas, MDPs y CSP

4.1. Redes Bayesianas

Definición 4.1.1 (Red Bayesiana). Una red bayesiana es un grafo acíclico dirigido (DAG) donde:

- Cada nodo representa una variable aleatoria.
- Los arcos representan dependencias directas.
- Cada nodo X_i tiene una CPT: $P(X_i \mid \text{Parents}(X_i))$.

4.1.1. Semántica

La distribución conjunta se factoriza como:

$$P(X_1, \dots, X_n) = \prod_{i=1}^n P(X_i \mid \text{Parents}(X_i))$$

4.1.2. Inferencia Exacta

- **Enumeración:** suma sobre todas las variables ocultas. Exponencial en el peor caso.
- **Eliminación de variables:** orden de eliminación importa para la eficiencia.

4.1.3. Inferencia Aproximada

- **Muestreo directo:** muestrea según el orden topológico de la red.
- **Muestreo por rechazo:** descarta muestras inconsistentes con la evidencia.
- **Ponderación por verosimilitud:** fija la evidencia, pondera las muestras.
- **MCMC / Gibbs Sampling:** muestrea cada variable condicionada al resto.

Ejemplo 4.1.1 (Red de alarma). Un sistema de alarma antirrobo se activa por un robo o (raramente) por un temblor. Dos vecinos, John y Mary, llaman si oyen la alarma, pero a veces confunden el teléfono con la alarma o no la oyen.

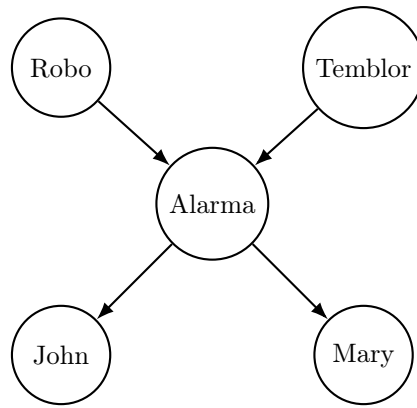


Figura 4.1: Red bayesiana Robo-Temblor-Alarma-John-Mary. Robo y Temblor son independientes a priori, pero se vuelven **dependientes** al condicionar sobre Alarma (*explaining away*); John y Mary son condicionalmente independientes dado Alarma.

$P(\text{Robo})$	$P(\text{Temblo})$	Robo	Temblo	$P(\text{Alarma})$
		V	V	0.95
		V	F	0.94
0.001	0.002	F	V	0.29
		F	F	0.001
			Alarma	$P(\text{Mary})$
			V	0.70
			F	0.01

Cuadro 4.1: CPTs de la red: la de Alarma tiene $2^2 = 4$ filas porque tiene 2 padres; John y Mary tienen 1 padre cada uno (Alarma), así que sus CPTs tienen 2 filas.

4.2. Procesos de Decisión de Markov (MDPs)

Definición 4.2.1 (MDP). Un MDP se define por la tupla (S, A, P, R, γ) :

- S : conjunto de estados
- A : conjunto de acciones
- $P(s' \mid s, a)$: modelo de transición (probabilístico)
- $R(s, a, s')$: función de recompensa
- $\gamma \in [0, 1]$: factor de descuento

4.2.1. Propiedad de Markov

$$P(s_{t+1} \mid s_t, a_t) = P(s_{t+1} \mid s_0, a_0, \dots, s_t, a_t)$$

El futuro solo depende del estado presente, no del historial.

4.2.2. Política y Utilidad

- **Política** $\pi(s)$: función que asigna una acción a cada estado.
- **Utilidad** (retorno descontado): $U = \sum_{t=0}^{\infty} \gamma^t R_t$
- **Política óptima** π^* : maximiza la utilidad esperada.

4.2.3. Ecuación de Bellman

$$U^*(s) = \max_a \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma U^*(s')]$$

4.2.4. Algoritmos de Solución

- **Value Iteration**: actualiza $U(s)$ iterativamente hasta convergencia.
- **Policy Iteration**: alterna entre evaluación de política y mejora de política.

Ejemplo 4.2.1 (Grid world 4×3). El agente se mueve en una cuadrícula de 4×3 celdas. La celda $(2, 2)$ es un muro (no se puede entrar). Los estados terminales son $(4, 3)$ con recompensa $+1$ y $(4, 2)$ con recompensa -1 ; cada movimiento a un estado no terminal cuesta $R(s) = -0,04$. Las acciones son deterministas en este ejemplo simplificado ($\gamma = 1$).

			+1
			-1
inicio			

Figura 4.2: Grid world clásico 4×3 : la celda gris en $(2, 2)$ es un muro; las metas están en $(4, 3)$ ($+1$) y $(4, 2)$ (-1). Cada paso no terminal cuesta $R(s) = -0,04$, lo que incentiva llegar rápido a la meta positiva.

Ejemplo 4.2.2 (Un paso de Value Iteration). Supóngase que, en una iteración anterior, $U(3, 3) = 0,85$ (celda justo a la izquierda de la meta $+1$) y que desde $(3, 2)$ la acción *arriba* tiene efecto determinista hacia $(3, 3)$. Aplicando la ecuación de Bellman:

$$U(3, 2) \leftarrow R(3, 2) + \gamma \cdot U(3, 3) = -0,04 + 1 \cdot 0,85 = 0,81$$

En cada iteración, los valores altos cerca de la meta $+1$ se van propagando hacia atrás, celda por celda, hasta que todos los $U(s)$ convergen.

4.3. Ejercicios: Razonamiento bajo Incertidumbre

Nota 4.3.1. Repaso de probabilidad, redes bayesianas y MDPs.

Ejercicio 4.3.1. Dada la red bayesiana del ejemplo (Robo-Temblor-Alarma-John-Mary), calcula $P(\text{Robo} \mid \text{John} = V, \text{Mary} = V)$ usando enumeración.

Solución: Por la regla de la cadena sobre el orden topológico de la red:

$$P(b \mid j, m) = \alpha \sum_e P(b)P(e) \sum_a P(a \mid b, e) P(j \mid a) P(m \mid a)$$

Para $b = V$ (usando las CPTs de la clase anterior):

$$(e=V): 0,002 \cdot [0,95 \cdot 0,9 \cdot 0,7 + 0,05 \cdot 0,05 \cdot 0,01] = 0,002 \cdot 0,598525 = 0,0011971$$

$$(e=F): 0,998 \cdot [0,94 \cdot 0,9 \cdot 0,7 + 0,06 \cdot 0,05 \cdot 0,01] = 0,998 \cdot 0,59223 = 0,5910575$$

Sumando y multiplicando por $P(b) = 0,001$: $P(b, j, m) = 0,001 \cdot 0,5922546 = 0,00059225$.

Repitiendo el mismo cálculo para $b = F$ (con $P(a \mid \neg b, e)$ de la CPT) se obtiene $P(\neg b, j, m) = 0,00149188$.

Normalizando con $\alpha = 1/(0,00059225 + 0,00149188) \approx 479,8$:

$$P(\text{Robo} \mid j, m) \approx 0,284 \quad P(\neg \text{Robo} \mid j, m) \approx 0,716$$

A pesar de que John y Mary llamaron, la probabilidad de robo sigue siendo menor que 0,5: la tasa base de robos es tan baja ($P(\text{Robo}) = 0,001$) que dos llamadas, aunque son evidencia fuerte, no son suficientes para hacerlo más probable que improbable.

4.4. Problemas de Satisfacción de Restricciones (CSP)

Definición 4.4.1 (CSP). Un CSP se define por:

- $X = \{X_1, \dots, X_n\}$: variables
- $D = \{D_1, \dots, D_n\}$: dominios de cada variable
- C : conjunto de restricciones sobre subconjuntos de variables

La **solución** es una asignación completa y consistente.

4.4.1. Tipos de Restricciones

- **Unarias:** $X_i \in \{v_1, v_2\}$
- **Binarias:** $X_i \neq X_j$
- **Globales:** involucran más de 2 variables (ej. AllDiff)

4.4.2. Búsqueda con Retroceso (Backtracking)

Asigna valores a variables una por una; retrocede cuando se viola una restricción.

Heurísticas para mejorar el backtracking:

1. **MRV** (Minimum Remaining Values): elegir la variable con menos valores posibles.
2. **Degree heuristic**: variable con más restricciones sobre variables no asignadas.
3. **Least Constraining Value**: valor que deja más opciones para las demás variables.

Ejemplo 4.4.1 (Coloreado de un mapa). Cuatro regiones W, X, Y, Z deben colorearse con {rojo, verde, azul} de modo que regiones vecinas tengan colores distintos. Las adyacencias son: $W-X$, $W-Y$, $X-Y$, $X-Z$, $Y-Z$.

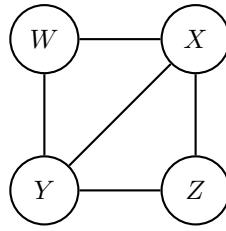


Figura 4.3: Grafo de restricciones del CSP: cada nodo es una variable (región) y cada arista es una restricción binaria de desigualdad con su vecina. X y Y tienen grado 3 (las más restringidas); son buenas candidatas para asignar primero según la heurística de grado.

Capítulo 5

Semana 5: CSP Avanzado y Agentes Lógicos

Nota 5.0.1 (Eventos esta semana). ■ Entrega Taller 2

- Examen 2

5.1. Problemas de Satisfacción de restricciones

5.1.1. Diagrama de Arbol

En la clase anterior se definió un CSP como una tripla (*variables, dominios, restricciones*) y se vió la **busqueda backtracking**, que es como un DFS + ordenamiento de variables + falla inmediata ante violación; junto con dos mejoras clave:

- **Filtrado** mediante *consistenciadearcos*; revisar si para cada valor de X, existe algún valor compatible en Y que detecta fallas inevitables antes que la simple revisión forward.
- **Ordenamiento** MRV (*Minimum Remaining Values*) para variables y LCV (*Least constraining value*) para valores

Ejemplo 5.1.1. Con dominios iniciales rojo,verde,azul y restricciones de desigualdad entre regiones vecinas, al forzar consistencia de arcos sobre el grafo WA NT SA Q NSW V se observa el efecto en cadena: Si el dominio de X pierde un valor, todos los vecinos de X se deben volver a

Arco revisado	Resultado
V → NSW	Consistente
SA → NSW	Consistente
NSW → SA	Inconsistente, se elimina azul del dominio de NSW
V → NSW	Como NSW cambi+o, se vuelve a ervisar se elimina rojo de V

Cuadro 5.1: Efecto en cadena al forzar consistencia de arcos.

revisar, porque una restriccion que antes era satisfacible puede dejar de serlo

Forzar consistencia de arcos sobre todo el CSP **no garantiza** encontrar la solución. al terminar el proceso puede que a quede exactamente una solución, b queden multiples soluciones sin decidir entre ellas o c el problema no tenga soluci´n y el algoritmo no lo sepa. Por eso la consistencia se arcos se corre dentro de una busqueda backtracking no en lugar de ella.

5.1.2. k-consistencia

La consistencia de arcos es apenas un caso particular de una idea más general, ir aumentando el grado de consistencia que se exige, de nodos a arcos, de arcos a trios de variables y así sucesivamente.

Definición 5.1.1. Un CSP es k -consistente si, para cualquier conjunto de $k-1$ variables y cualquier asignación consistente a esas $k-1$ variables, siempre existe un valor consistente para extender la asignación a una k -ésima variable cualquiera.

Nivel	Que garantiza
1-consistencia (consistencia de nodos)	Cada dominio ya respeta las restricciones unarias del nodo
2-consistencia (consistencia de arcos)	Toda asignación consistente a un nodo se extiende al otro
3-consistencia (consistencia de caminos)	Todo par de nodos consistentes se puede extender a un tercero
k -consistencia	Se cumple simultáneamente para $1, 2, \dots, k$

A mayor k , el filtrado poda más el árbol de búsqueda, pero también es **mas caro de computar**. No hay almuerzo gratis, es un tradeoff entre tiempo invertido en propagación de restricciones y tiempo invertido en backtracking.

5.1.3. Forward Checking

Es una alternativa más barata que forzar consistencia de arcos completa. Cada vez que se asigna un valor a X_i , se recorren las variables **vecinas no asignadas** de X_i y se elimina de sus dominios cualquier valor inconsistente con la asignación recién hecha.

Definición 5.1.2 (Forward Checking). Al asignar $X_i \leftarrow v$, para cada variable X_j vecina de X_i y aún sin asignar, eliminar de D_j todo valor que no sea consistente con $X_i = v$. Si algún D_j queda vacío, se detecta el fallo inmediatamente sin necesidad de llegar a esa variable.

La diferencia con la consistencia de arcos (AC-3) es que forward checking **solo revisa los arcos que salen de la variable recién asignada**, una sola vez. No propaga el efecto en cadena hacia el resto del grafo como sí lo hace AC-3 (que vuelve a revisar los vecinos de un vecino cuando su dominio cambia). Por eso forward checking es más barato pero detecta menos inconsistencias por adelantado; es un punto intermedio entre backtracking simple (0 chequeo hacia adelante) y AC-3 completo (propagación total).

5.1.4. Estructura del grafo de restricciones

Cuando el grafo de restricciones de un CSP es un **árbol** (sin ciclos), el problema se puede resolver en tiempo $\mathcal{O}(nd^2)$, sin ningún backtracking.

Definición 5.1.3 (CSP con estructura de árbol). 1. Elegir una variable como raíz y ordenar el resto en un orden topológico $X_1, \dots, X_n$ tal que cada variable aparezca después de su padre en el árbol.

2. Recorrer las variables **de atrás hacia adelante** (j de n a 2) y forzar consistencia de arco entre X_j y su padre $\text{Padre}(X_j)$, eliminando del dominio del padre los valores que ya no tengan soporte.

3. Recorrer las variables **de adelante hacia atrás** (j de 1 a n) asignando a cada X_j cualquier valor de su dominio consistente con el valor ya asignado a $\text{Padre}(X_j)$.

Como el grafo quedó dirigido y arco-consistente antes de asignar, el segundo recorrido nunca falla: nunca hay que retroceder.

La mayoría de los CSP reales no son árboles, pero se pueden **acercar** a esa estructura:

- **Cutset conditioning**: encontrar un conjunto pequeño de variables (*cutset*) tal que, al eliminarlas del grafo, lo que queda es un árbol. Se prueban todas las asignaciones consistentes posibles para el cutset y, para cada una, se resuelve el árbol restante en $\mathcal{O}(nd^2)$. Si el cutset tiene tamaño c , el costo total es $\mathcal{O}(d^c \cdot (n - c)d^2)$: exponencial en c , pero manejable si c es pequeño respecto a n .
- **Tree decomposition**: agrupar variables en subproblemas conectados en forma de árbol (cada subproblema se resuelve por separado) para luego combinar soluciones consistentes entre subproblemas vecinos.

5.1.5. Búsqueda local para CSP: Min-conflicts

A diferencia de backtracking (que construye la asignación variable por variable), min-conflicts parte de una **asignación completa** (posiblemente inconsistente) y la va reparando.

Definición 5.1.4 (Min-conflicts). 1. Generar una asignación inicial completa, asignando a cada variable un valor cualquiera de su dominio (puede violar restricciones).

2. Repetir hasta encontrar una solución o alcanzar un número máximo de pasos:

- a) Elegir al azar una variable X_i que esté en conflicto (viola alguna restricción).
- b) Reasignarle el valor de su dominio que **minimice el número de conflictos** con las demás variables (rompiendo empates al azar).

Ejemplo 5.1.2 (Min-conflicts en n-reinas). Con las 8 reinas ya puestas una por columna (asignación inicial con conflictos), en cada paso se elige una reina que ataque a otra y se mueve dentro de su columna a la fila con menos reinas atacándola. En la práctica, resuelve instancias de miles de reinas en muy pocos pasos.

Es muy eficiente en la práctica incluso para CSPs grandes, pero **no garantiza completitud**: puede quedar atascado y nunca encontrar una solución aunque exista (no hay noción de retroceso ni de dominios que se agotan).

5.2. Ejercicio: Coloreado del mapa de Australia

Ejercicio 5.2.1. Aplicar backtracking con MRV (y desempate por grado) y propagación tipo forward checking al problema de colorear el mapa de Australia con 3 colores {rojo, verde, azul}, exigiendo que regiones vecinas tengan colores distintos. Regiones: WA, NT, SA, Q, NSW, V, T (Tasmania es una isla, no tiene vecinos). Adyacencias: WA-NT, WA-SA, NT-SA, NT-Q, SA-Q, SA-NSW, SA-V, Q-NSW, NSW-V.

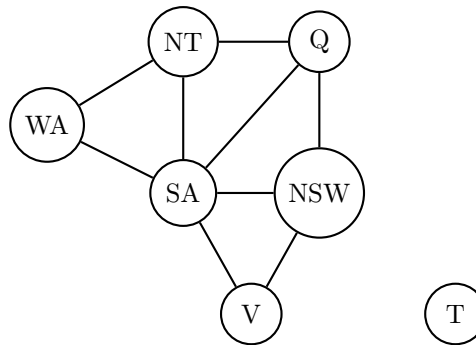


Figura 5.1: Grafo de restricciones del mapa de Australia: SA es vecina de 5 regiones (grado 5), la más restringida del grafo; T (Tasmania) no tiene aristas porque es una isla sin vecinos.

Solución:

Todas las variables (salvo T) parten con $|D| = 3$: hay empate en MRV, así que se desempata con la **heurística de grado** (más restricciones sobre variables sin asignar todavía). SA tiene grado 5, el más alto de todo el grafo, así que se asigna primero.

Paso	Asignación	Efecto de la propagación
1	SA = rojo	Se elimina rojo de WA, NT, Q, NSW, V
2	Q = verde	Se elimina verde de NT y de NSW
3	NT = azul	Se elimina azul de WA
4	WA = verde	Sin vecinos nuevos por revisar
5	NSW = azul	Se elimina azul de V
6	V = verde	Sin vecinos nuevos por revisar
7	T = rojo	T no tiene restricciones, cualquier color sirve

Cuadro 5.2: Traza de backtracking + MRV + forward checking sobre el mapa de Australia.

En cada paso, después de asignar, la variable que queda con el dominio más pequeño (a veces de tamaño 1) es la siguiente elegida por MRV, así que no hace falta romper más empates. La propagación evita que en este caso se necesite retroceder: ningún dominio queda vacío en el camino.

Solución final: SA = rojo, Q = verde, NT = azul, WA = verde, NSW = azul, V = verde, T = rojo (o cualquier color, T no tiene vecinos).

5.3. Agentes Lógicos: Introducción

5.3.1. Agentes basados en conocimiento

Un agente basado en conocimiento mantiene una **base de conocimiento** (KB): un conjunto de oraciones en algún lenguaje de representación que expresan hechos sobre el mundo. El agente decide qué acción tomar consultando (ASK) e incorporando percepciones nuevas (TELL) a la KB, usando un **motor de inferencia**.

Definición 5.3.1 (Agente basado en conocimiento). En cada ciclo el agente: (1) le dice (TELL) a la KB lo que percibió, (2) le pregunta (ASK) cuál es la mejor acción dado lo que sabe, y (3) le dice a la KB qué acción ejecutó. Todo el conocimiento sobre el dominio vive en la KB, no en el código del agente, así que se puede razonar y no solo reaccionar.

$KB \vdash \alpha$ (la KB **deriva** α sintácticamente, con un procedimiento de inferencia)

Un procedimiento de inferencia es **sólido** (*sound*) si solo deriva oraciones que realmente son consecuencia lógica de la KB, y es **completo** si es capaz de derivar toda oración que sea consecuencia lógica.

5.3.2. Lógica proposicional

Sintaxis: oraciones atómicas (símbolos proposicionales, cada uno verdadero o falso) combinadas con conectivos lógicos $\neg$ (negación), $\wedge$ (conjunción), $\vee$ (disyunción), $\Rightarrow$ (implicación), $\Leftrightarrow$ (bicondicional), para formar oraciones complejas.

Semántica: un **modelo** es una asignación de verdadero/falso a cada símbolo proposicional. Dado un modelo, el valor de verdad de cualquier oración se calcula recursivamente con las tablas de verdad de los conectivos.

- Una oración es **válida** (tautología) si es verdadera en **todos** los modelos (ej. $P \vee \neg P$).
- Una oración es **satisfacible** si es verdadera en **al menos un** modelo.
- Una oración es **insatisfacible** si no es verdadera en ningún modelo (ej. $P \wedge \neg P$).
- **Consecuencia lógica (entailment):** $KB \models \alpha$ si en todo modelo donde KB es verdadera, α también lo es. Equivalentemente, $KB \models \alpha$ si y solo si $KB \wedge \neg \alpha$ es insatisfacible.

Ejemplo 5.3.1 (Model checking). Para decidir si $KB \models \alpha$ por enumeración de modelos: se listan todas las combinaciones de verdad de los símbolos que aparecen en KB y en α (tabla de verdad completa), y se revisa que en toda fila donde KB sea verdadera, α también lo sea. Es correcto pero exponencial en el número de símbolos.

5.3.3. Formas normales: CNF y DNF

- **CNF** (Forma Normal Conjuntiva): conjunción de cláusulas, donde cada cláusula es una disyunción de literales. Es la forma que usan la mayoría de los algoritmos de inferencia (resolución, DPLL, etc.).
- **DNF** (Forma Normal Disyuntiva): disyunción de términos, donde cada término es una conjunción de literales.

Pasos para convertir cualquier oración a CNF:

1. Eliminar $\Leftrightarrow$: reemplazar $\alpha \Leftrightarrow \beta$ por $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$.
2. Eliminar $\Rightarrow$: reemplazar $\alpha \Rightarrow \beta$ por $\neg \alpha \vee \beta$.
3. Meter las negaciones hacia adentro usando De Morgan ($\neg(\alpha \wedge \beta) \equiv \neg \alpha \vee \neg \beta$, $\neg(\alpha \vee \beta) \equiv \neg \alpha \wedge \neg \beta$) y eliminar dobles negaciones.
4. Distribuir $\vee$ sobre $\wedge$ hasta que quede una conjunción de disyunciones de literales.

5.3.4. El mundo del Wumpus

Ambiente clásico para introducir agentes lógicos: una cueva en cuadrícula (típicamente 4×4), con un Wumpus que mata al agente si entra en su casilla, pozos que lo hacen caer, y oro que debe recoger y sacar. El agente empieza en (1, 1) y solo percibe su casilla actual.

- **Percepciones:** *stench* (hedor, si el Wumpus está en una casilla adyacente), *breeze* (brisa, si hay un pozo adyacente), *glitter* (brillo, si hay oro en la casilla), *bump* (chocó con pared), *scream* (grito, al matar al Wumpus con una flecha).

- **Acciones:** avanzar, girar izquierda/derecha, agarrar, disparar, subir (para salir de la cueva).

seguro	?		
inicio	seguro brisa	?	

Figura 5.2: Estado de conocimiento tras percibir en (1,1) sin hedor ni brisa (por lo que (1,2) y (2,1) son seguras), y luego en (2,1) brisa sin hedor: hay un pozo en (2,2) o en (3,1), pero no se sabe cuál con la información acumulada hasta ahora.

Ejemplo 5.3.2 (Inferencia en el mundo del Wumpus). Si en (1,1) el agente no percibe ni hedor ni brisa, la regla $Breeze_{(x,y)} \Leftrightarrow \bigvee Pit_{vecino}$ (y la equivalente para *Stench* y el Wumpus) permite concluir que **ninguna** casilla vecina, (1,2) y (2,1), tiene pozo ni Wumpus: son seguras para avanzar. Si luego en (2,1) el agente sí percibe brisa (pero no hedor), infiere que hay un pozo en (3,1) o en (2,2), pero **no sabe en cuál** de los dos sin más información; ese es justamente el tipo de razonamiento bajo información parcial que la KB tiene que resolver combinando todas las percepciones acumuladas, no solo la última.

Parte III

Lógica y Representación del Conocimiento

Capítulo 6

Semana 6: Lógica y Representación del Conocimiento

6.1. Agentes Lógicos II: Inferencia Proposicional

6.1.1. Reglas de Inferencia

Regla	Forma
Modus Ponens	$\alpha \Rightarrow \beta, \alpha \vdash \beta$
Modus Tollens	$\alpha \Rightarrow \beta, \neg\beta \vdash \neg\alpha$
Resolución	$(\ell_1 \vee \dots \vee \ell_k), (\neg\ell_1 \vee m_1 \vee \dots) \vdash (m_1 \vee \dots)$

6.1.2. Resolución

Algoritmo de inferencia completo para lógica proposicional en CNF:

1. Convertir $KB \wedge \neg\alpha$ a CNF.
2. Aplicar resolución hasta obtener la cláusula vacía (contradicción) o no poder seguir.
3. Si se obtiene contradicción $\Rightarrow KB \models \alpha$.

6.1.3. DPLL y WalkSAT

- **DPLL**: backtracking completo sobre asignaciones de verdad, con propagación de cláusulas unitarias.
- **WalkSAT**: búsqueda local incompleta, eficaz en la práctica para instancias satisfacibles.

6.1.4. Ejemplo: Refutación por Resolución

Ejemplo 6.1.1 (Refutación). Sea $KB = \{P, P \Rightarrow Q, Q \Rightarrow R\}$ y se quiere probar $KB \models R$. Se convierte $KB \wedge \neg R$ a CNF: $\{P, \neg P \vee Q, \neg Q \vee R, \neg R\}$, y se aplica resolución hasta obtener la cláusula vacía $\square$ (una contradicción), lo que confirma el entailment.

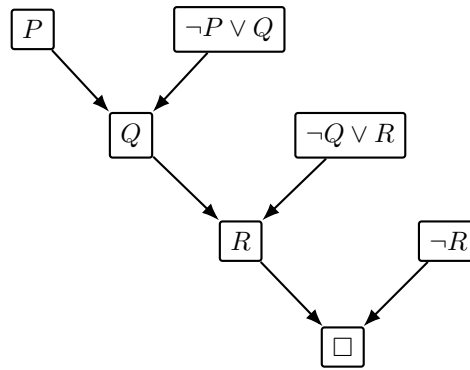


Figura 6.1: Árbol de refutación por resolución: cada nodo se obtiene combinando los dos que apuntan hacia él; llegar a la cláusula vacía $\square$ demuestra que $KB \wedge \neg R$ es insatisfacible, es decir, $KB \models R$.

Ejercicio 6.1.1. Sea $KB = \{A \vee B, \neg A \vee C, \neg B \vee C, \neg C \vee D\}$. Se quiere probar $KB \models D$ por refutación.

1. Escriba la cláusula que se agrega a KB al negar la meta.
2. Aplique resolución sobre $A \vee B$ y $\neg A \vee C$, y por separado sobre $\neg B \vee C$, para derivar una cláusula con únicamente C .
3. Combine el resultado anterior con $\neg C \vee D$ y con la meta negada para llegar a la cláusula vacía.
4. Dibuje el árbol de refutación completo, análogo al de la Figura anterior.

6.2. Lógica de Primer Orden (LPO)

6.2.1. Sintaxis

Elementos de LPO:

- **Constantes:** *Juan, UNIANDES, ...*
- **Variables:** *x, y, z, ...*
- **Predicados:** *Estudiante(x), Mayor(x, y)*
- **Funciones:** *Padre(x), Suma(x, y)*
- **Cuantificadores:** $\forall$ (universal), $\exists$ (existencial)
- **Conectivos:** $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$

6.2.2. Semántica

Un **modelo** en LPO especifica:

- El dominio de objetos
- La interpretación de constantes, predicados y funciones

6.2.3. Cuantificadores

$$\forall x P(x) \equiv \neg \exists x \neg P(x)$$

$$\exists x P(x) \equiv \neg \forall x \neg P(x)$$

Nota 6.2.1. $\forall x (P(x) \Rightarrow Q(x))$ es diferente de $\forall x (P(x) \wedge Q(x))$. El cuantificador universal normalmente usa $\Rightarrow$; el existencial usa $\wedge$.

6.2.4. Ejemplo: Modelo en LPO

Ejemplo 6.2.1 (Modelo simple). Sea el dominio $D = \{\text{Juan, Maria, Pedro}\}$ y el predicado binario $\text{Padre}(x, y)$ (x es padre de y). La interpretación de la Figura siguiente satisface $\text{Padre}(\text{Juan, Maria})$ y $\text{Padre}(\text{Juan, Pedro})$, y ninguna otra pareja.

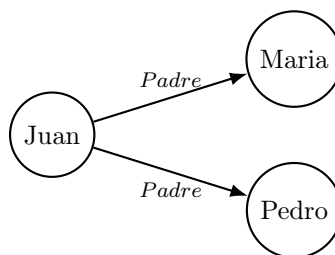


Figura 6.2: Modelo con dominio $\{\text{Juan, Maria, Pedro}\}$: cada flecha es una pareja (x, y) para la cual $\text{Padre}(x, y)$ es verdadero en este modelo. Cambiar las flechas cambia el modelo sin cambiar la sintaxis de la oración.

Ejercicio 6.2.1. Usando los predicados $\text{Estudiante}(x)$, $\text{Curso}(y)$, $\text{Aprueba}(x, y)$ y $\text{Certificado}(x, y)$, traduzca a LPO:

1. Todo estudiante que aprueba un curso recibe un certificado de ese curso.
2. Existe al menos un estudiante que no ha aprobado ningún curso.
3. Ningún estudiante aprueba todos los cursos.

Para cada oración, indique si el cuantificador principal es $\forall$ o $\exists$ y justifique si el conectivo correcto dentro de su alcance es $\Rightarrow$ o $\wedge$.

6.3. LPO e Inferencia I

6.3.1. Bases de Conocimiento en LPO

Las bases de conocimiento en LPO pueden expresar relaciones complejas de forma compacta, a diferencia de la lógica proposicional.

6.3.2. Unificación

Para aplicar reglas de inferencia en LPO se necesita **unificar** expresiones:

$$\text{UNIFY}(\alpha, \beta) = \theta \text{ tal que } \text{SUBST}(\theta, \alpha) = \text{SUBST}(\theta, \beta)$$

Algoritmo: recorre la estructura de ambas expresiones, construyendo sustituciones. Devuelve *fallo* si no hay unificador.

6.3.3. Inferencia con Cuantificadores

- **Instanciación universal:** $\forall x \alpha(x) \vdash \alpha(c)$ para cualquier constante c .
- **Instanciación existencial:** $\exists x \alpha(x) \vdash \alpha(k)$ donde k es una nueva constante de Skolem.
- **Skolemización:** eliminar cuantificadores existenciales introduciendo constantes/funciones de Skolem.

6.3.4. Forma Normal de Clausura (CNF en LPO)

Pasos para convertir a CNF:

1. Eliminar bicondicionales e implicaciones.
2. Mover $\neg$ hacia adentro (De Morgan, reglas de cuantificadores).
3. Estandarizar variables.
4. Skolemizar.
5. Eliminar cuantificadores universales.
6. Distribuir $\vee$ sobre $\wedge$.

6.3.5. Ejemplo: Unificación

Ejemplo 6.3.1 (Unificación de dos átomos). Para unificar $Padre(x, Juan)$ con $Padre(Pedro, y)$, el algoritmo recorre ambas expresiones posición por posición: en la primera posición x debe sustituirse por Pedro; en la segunda, y debe sustituirse por Juan. El unificador más general es $\theta = \{x/Pedro, y/Juan\}$.

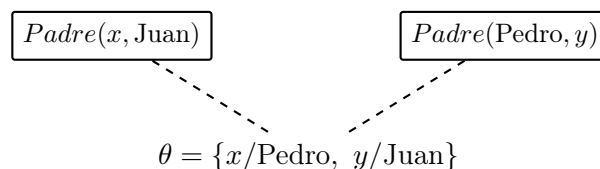


Figura 6.3: Unificación de $Padre(x, Juan)$ y $Padre(Pedro, y)$: aplicar θ a ambas expresiones produce la misma expresión, $Padre(Pedro, Juan)$.

- Ejercicio 6.3.1.**
1. Encuentre, si existe, el unificador más general de $Quiere(x, Helado(x))$ y $Quiere(Juan, Helado(Vainilla))$.
 2. Encuentre el unificador de $Hermano(x, y)$ y $Hermano(Juan, y)$. ¿Por qué es necesario estandarizar variables antes de unificar dos cláusulas que provienen de la misma regla?
 3. Convierta a CNF: $\forall x (Estudiante(x) \Rightarrow \exists y (Curso(y) \wedge Inscrito(x, y)))$, indicando en qué paso aparece la función de Skolem y qué representa.

6.4. Inferencia en LPO II

6.4.1. Encadenamiento hacia adelante (Forward Chaining)

Aplica reglas de la forma $P_1 \wedge \dots \wedge P_n \Rightarrow Q$ para derivar nuevos hechos hasta alcanzar la meta.

- Completo para bases de conocimiento de **cláusulas de Horn**.
- Dirigido por los datos.

6.4.2. Encadenamiento hacia atrás (Backward Chaining)

Parte de la meta y trabaja hacia atrás buscando hechos o submetas.

- Completo para cláusulas de Horn.
- Base de Prolog.
- Puede ciclar si no se controla.

6.4.3. Resolución en LPO

Generaliza la resolución proposicional usando unificación:

$$\frac{\ell_1 \vee \dots \vee \ell_k \quad \neg m_1 \vee \dots \vee \neg m_j}{\text{SUBST}(\theta, \ell_1 \vee \dots \vee m_j)}$$

donde $\theta = \text{UNIFY}(\ell_i, m_k)$.

- **Refutación por resolución:** completa para LPO (Teorema de Completitud de Gödel).

6.4.4. Ejemplo: El caso de Nono (¿Es West un criminal?)

Ejemplo 6.4.1 (Base de conocimiento). 1. $\text{American}(x) \wedge \text{Weapon}(y) \wedge \text{Sells}(x, y, z) \wedge \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$

2. $\text{Owns}(\text{Nono}, M1)$

3. $\text{Missile}(M1)$

4. $\text{Missile}(x) \wedge \text{Owns}(\text{Nono}, x) \Rightarrow \text{Sells}(\text{West}, x, \text{Nono})$

5. $\text{Missile}(x) \Rightarrow \text{Weapon}(x)$

6. $\text{Enemy}(x, \text{America}) \Rightarrow \text{Hostile}(x)$

7. $\text{American}(\text{West})$

8. $\text{Enemy}(\text{Nono}, \text{America})$

Meta: probar $\text{Criminal}(\text{West})$.

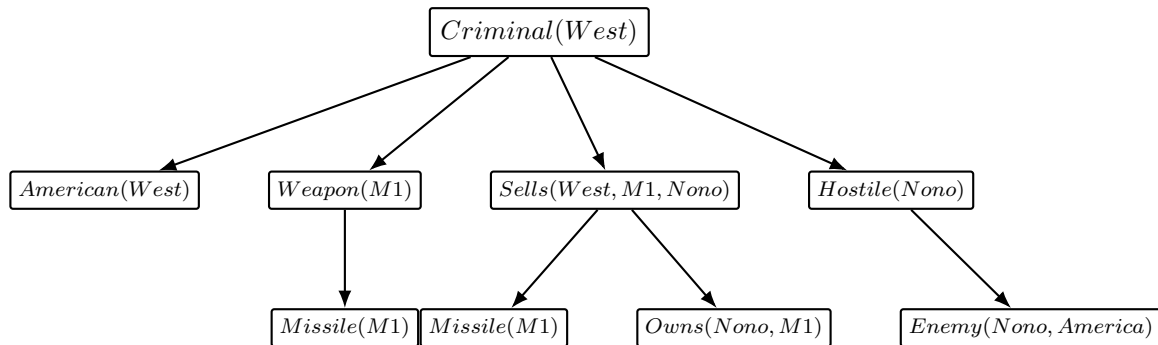


Figura 6.4: Árbol de encadenamiento hacia atrás para $\text{Criminal}(\text{West})$: los cuatro hijos de la raíz son las **cuatro submetas** que exige la conjunción de la regla 1 (todas deben probarse), y $\text{Missile}(M1)$ y $\text{Owns}(\text{Nono}, M1)$ son las dos submetas que exige la regla 4. Cada hoja es un hecho que ya está en la KB (reglas 2, 3, 7 y 8).

Submeta	Regla utilizada	Qué se obtiene
<i>Criminal(West)</i>	Regla 1	Se descompone en 4 submetas (AND)
<i>American(West)</i>	Regla 7 (hecho)	Submeta resuelta directamente
<i>Weapon(M1)</i>	Regla 5	Se reduce a la submeta <i>Missile(M1)</i>
<i>Missile(M1)</i>	Regla 3 (hecho)	Submeta resuelta directamente
<i>Sells(West, M1, Nono)</i>	Regla 4	Se reduce a <i>Missile(M1) ∧ Owns(Nono, M1)</i>
<i>Owns(Nono, M1)</i>	Regla 2 (hecho)	Submeta resuelta directamente
<i>Hostile(Nono)</i>	Regla 6	Se reduce a <i>Enemy(Nono, America)</i>
<i>Enemy(Nono, America)</i>	Regla 8 (hecho)	Submeta resuelta directamente

Cuadro 6.1: Traza del encadenamiento hacia atrás: en cada fila se indica qué regla dispara la reducción y qué submeta nueva (o hecho) se obtiene, hasta que todas las hojas quedan probadas y *Criminal(West)* queda demostrado.

Ejercicio 6.4.1. 1. Usando la misma KB, construya la traza de **encadenamiento hacia adelante**: parta de los hechos (reglas 2, 3, 7, 8) y muestre, en una tabla como la anterior, en qué orden se disparan las reglas 5, 6, 4 y 1 hasta derivar *Criminal(West)*.

2. Compare las dos trazas: ¿por qué el encadenamiento hacia atrás solo explora los hechos relevantes para la meta, mientras que el hacia adelante puede derivar hechos que nunca se usan?
3. Dado el hecho adicional *Enemy(Nono, UK)* (en vez de *America*), ¿sigue siendo posible probar *Criminal(West)* con esta KB? Justifique cuál regla falla.

Ejercicio 6.4.2 (Resolución en LPO). Sean las cláusulas

$$\neg \text{Ama}(x, \text{Padre}(x)) \vee \text{Feliz}(x) \quad \text{y} \quad \text{Ama}(\text{Juan}, \text{Padre}(\text{Juan})).$$

1. Encuentre el unificador θ entre los literales complementarios.
2. Aplique $\text{SUBST}(\theta, \cdot)$ a la cláusula resolvente resultante.
3. ¿Qué hecho queda demostrado?

Parte IV

Planeación

Capítulo 7

Semana 7: Inferencia en LPO y Planeación

Eventos esta semana

- Publicación Taller 3

7.1. Inferencia en LPO: Cierre

Nota

Repaso de resolución en LPO con ejemplos. Transición hacia planeación.

7.2. Planeación: Definición y Algoritmos Generales

7.2.1. ¿Qué es la Planeación?

Un agente planificador construye una **secuencia de acciones** para lograr un objetivo en un entorno modelado.

Definición 7.2.1 (Problema de Planeación STRIPS/PDDL).
literales positivos/negativos).

- **Estado:** conjunción de fluen-
- **Acción:** definida por precondiciones y efectos.
- **Meta:** conjunción de fluentes que deben ser verdaderos.

7.2.2. Representación PDDL

Listing 7.1: Ejemplo de acción en PDDL

```
1 (:action Fly
2   :parameters (?p - Plane ?from ?to - Airport)
3   :precondition (and (At ?p ?from) (Plane ?p)
4                     (Airport ?from) (Airport ?to))
5   :effect (and (At ?p ?to) (not (At ?p ?from))))
```

7.2.3. Algoritmos de Búsqueda para Planeación

- **Búsqueda hacia adelante** (*progression*): empieza en el estado inicial y aplica acciones.
- **Búsqueda hacia atrás** (*regression*): empieza desde la meta y aplica acciones inversas.
- **SATPLAN**: codifica el problema de planeación como SAT y usa un solucionador SAT.

7.3. Planeación: Heurísticas y Planeación Jerárquica

7.3.1. Heurísticas para Planeación

Problema Relajado (ignore preconditions / ignore delete lists)

- **Ignore preconditions**: todas las acciones son aplicables. Heurística: número mínimo de acciones para satisfacer la meta.
- **Ignore delete lists**: no se eliminan fuentes. Problema de planeación relajado resoluble en tiempo polinomial.

Planning Graph y Heurística FF

Un **grafo de planeación** alterna capas de literales y acciones:

- Capa de literales S_i : literales alcanzables en i pasos.
- Capa de acciones A_i : acciones aplicables en el paso i .

La heurística FF usa el plan del grafo relajado como estimador.

7.3.2. Planeación Jerárquica (HTN)

Hierarchical Task Network (HTN): descompone tareas abstractas en subtareas más concretas.

- **Tareas primitivas**: acciones directamente ejecutables.
- **Tareas abstractas**: se descomponen mediante métodos.
- **Ventaja**: permite incorporar conocimiento de dominio para guiar la búsqueda.

7.4. Planeación en Ambientes No Deterministas

7.4.1. Ambientes No Deterministas y Condicionales

Cuando las acciones tienen efectos inciertos, se necesitan **planes condicionales** que incluyan ramificaciones según el resultado de las acciones.

- **Plan de búsqueda AND-OR**: los nodos AND representan contingencias del entorno; los nodos OR representan elecciones del agente.
- Un plan condicional es una solución si cubre todas las ramas.

7.4.2. Planeación Parcialmente Observable

Con observabilidad parcial el agente mantiene un **estado de creencia** (belief state) y ejecuta acciones de recopilación de información.

7.4.3. Planeación Continua y de Ejecución

- **Monitoreo de ejecución:** detectar discrepancias entre el plan y la realidad.
- **Replaneación:** adaptar el plan ante fallos o nueva información.
- **Agentes reactivos-deliberativos:** combinan planeación a largo plazo con respuesta reactiva.

Parte V

IA Moderna, Aplicaciones y Ética

Capítulo 8

Semana 8: Aplicaciones, Ética y Repaso Final

Eventos esta semana

- Entrega Taller 3
- Examen Final

8.1. Aplicaciones en IA

8.1.1. Machine Learning

Paradigma donde los sistemas **aprenden de datos** en lugar de ser programados explícitamente.

- **Aprendizaje supervisado:** clasificación, regresión.
- **Aprendizaje no supervisado:** clustering, reducción de dimensionalidad.
- **Aprendizaje por refuerzo (RL):** aprender mediante interacción con el entorno y recompensas.

8.1.2. Deep Learning

Redes neuronales profundas con múltiples capas de abstracción.

- CNN (Convolutional Neural Networks): visión computacional.
- RNN / Transformers: procesamiento de lenguaje natural (NLP).
- LLMs: modelos de lenguaje de gran escala (ej. GPT, Claude).

8.1.3. Áreas de Aplicación

- **NLP:** traducción automática, chatbots, análisis de sentimientos.
- **Visión computacional:** detección de objetos, reconocimiento facial, diagnóstico médico por imágenes.
- **Robótica:** percepción, planeación de movimiento, manipulación.

8.2. Ética en IA y Aplicaciones Futuras

8.2.1. Riesgos y Desafíos Éticos

- **Sesgo algorítmico:** sistemas entrenados con datos sesgados perpetúan o amplifican discriminación.
- **Privacidad:** recolección masiva de datos personales.
- **Seguridad:** ataques adversariales, manipulación de sistemas autónomos.
- **Desplazamiento laboral:** automatización de tareas cognitivas y manuales.
- **Concentración de poder:** dominio de pocas empresas sobre infraestructura de IA.

8.2.2. Marcos Regulatorios

- **AI Act** (Unión Europea): regulación por niveles de riesgo.
- Lineamientos de uso de IA generativa en contextos académicos.

8.2.3. Niveles de Uso de IA Generativa (Uniandes)

Nivel	Uso	Descripción
1	Sin IA generativa	No debe usarse en ningún aspecto
2	Generar/estructurar ideas	No incluir material generado en el trabajo
3	Editar y refinar	Explicar uso en nota al pie o anexo
4	Completar tareas (revisión humana)	Distinguir material IA; incluir transcripción de prompts
5	Uso pleno	Sin restricciones de distinción

8.2.4. Futuro de la IA

8.3. Repaso para Examen Final

Nota

Repaso de todos los temas del curso en preparación para el **Examen Final**. El examen cubre: búsqueda, incertidumbre, CSP, lógica y planeación.

8.3.1. Resumen de Temas

1. **Agentes inteligentes:** PEAS, tipos de agente, naturaleza del entorno.
2. **Búsqueda no informada:** BFS, DFS, IDDFS, UCS.
3. **Búsqueda informada:** A*, IDA*, RBFS; admisibilidad y consistencia.
4. **Búsqueda local:** Hill climbing, simulated annealing, algoritmos genéticos.
5. **Búsqueda adversaria:** Minimax, poda α - β , MCTS.
6. **Incertidumbre:** probabilidad, Bayes, redes bayesianas, inferencia, MDPs.
7. **CSP:** backtracking, AC-3, MRV, min-conflicts.
8. **Lógica:** proposicional, LPO, resolución, encadenamiento.
9. **Planeación:** PDDL, búsqueda en espacio de planes, heurísticas, HTN.